



SECJ3483: Web Technology

Individual Assignment Report- DHL Project

(Web Application & Robotic Process Automation (RPA))

SCENARIO 1 : AUTOMATED SOP INGESTION SYSTEM FOR DHL LOGISTICS

Name	Matric Number
Nur Iman Farisya Binti Mohammad Hazlin	A23MJ5030

SECTION 1 : INTRODUCTION AND SCENARIO DESCRIPTION

1.1 Document Ingestion Challenges in Modern Logistics

Standardizing compliance workflows across international networks like DHL requires real-time management of regional Standard Operating Procedures (SOPs) governing hazardous cargo, cold-chain limits, and customs rules. Processing this technical data manually introduces severe transcription lag, errors, and administrative overhead, preventing logistics teams from adapting quickly to shifting global supply chain directives.

1.2 Selection of Assignment Parameter: Scenario 1

To address these bottlenecks, this project implements **Scenario 1: The Automated DHL SOP Ingestion System**. This intelligent, multi-tier pipeline replaces manual ingestion to mitigate three core operational vulnerabilities:

- **Repository Pollution:** Prevents frontline operators from re-uploading identical or slightly modified procedures across isolated silos, eliminating data clutter.
- **Storage Degradation:** Eliminates redundant copies of long-form text procedures that cause database bloat and inflate infrastructure storage costs.
- **Analytical Disruption:** Removes duplicate records that skew compliance metrics, ensuring quality assurance officers maintain a single source of truth.

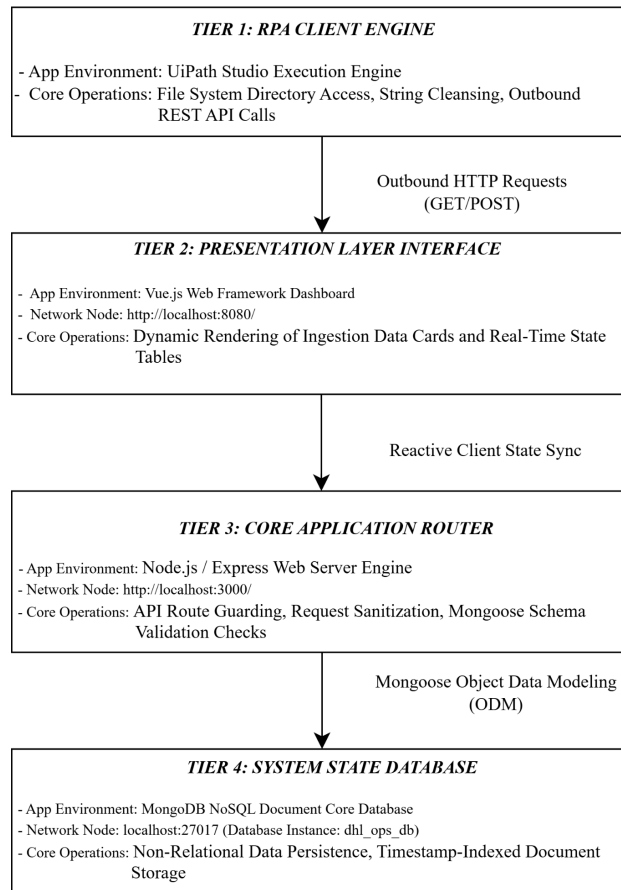
1.3 The Core Solution Architecture

To resolve these issues, the system establishes a decoupled 4-Tier Architecture connecting a desktop automation worker to a web API gateway. Moving away from rigid relational structures, the system utilizes a **Node.js/Express API routing server** paired with a flexible **MongoDB NoSQL database**.

A background robot orchestrated in **UiPath Studio** continuously polls local folders, extracts raw file text into memory, and triggers HTTP REST handshakes to validate content. By checking incoming payloads against existing MongoDB JSON collections in real-time, the API gateway blocks duplicate entries at the perimeter, guarantees data integrity, and streams clean operational metrics onto a reactive **Vue.js browser dashboard**.

SECTION 2: SYSTEM ARCHITECTURE DESIGN

To maximize stability and ensure a separation of concerns, the platform uses a decoupled 4-Tier Architecture. This modular design isolates file-system monitoring, logical validation, data storage, and graphical visualization into independent processing nodes that communicate exclusively via standard application layer protocols.



2.1 Layer Component Explanations

Tier 1: Robotic Process Automation (RPA) Client Engine (UiPath Studio) : Orchestrates file discovery by polling local folders, parses unformatted text files into memory strings, and executes HTTP REST api calls. It functions with zero database access privileges, routing all requests exclusively through the API gateway.

Tier 2: Presentation Layer Interface (Vue.js Web Application) : A reactive Single Page Application (SPA) deployed on Port 8080 that serves as the operational dashboard. It asynchronously consumes JSON payloads from the backend to dynamically update data tables and compliance metrics with zero page refreshes.

Tier 3: Core Application Router & Logic Gateway (Node.js & Express) : An asynchronous backend server running on Port 3000 that serves as the central logic gateway. It manages API

routing, validates inbound payloads, processes duplicate-checking evaluations, and enforces Mongoose validation schemas before allowing data mutations.

Tier 4: Persistent System Database Tier (MongoDB NoSQL Server) : A document data store running on Port 27017 inside the dhl_ops_db instance. It replaces rigid relational tables with flexible, high-speed JSON/BSON documents that are automatically indexed via system ObjectIDs and chronological timestamps.

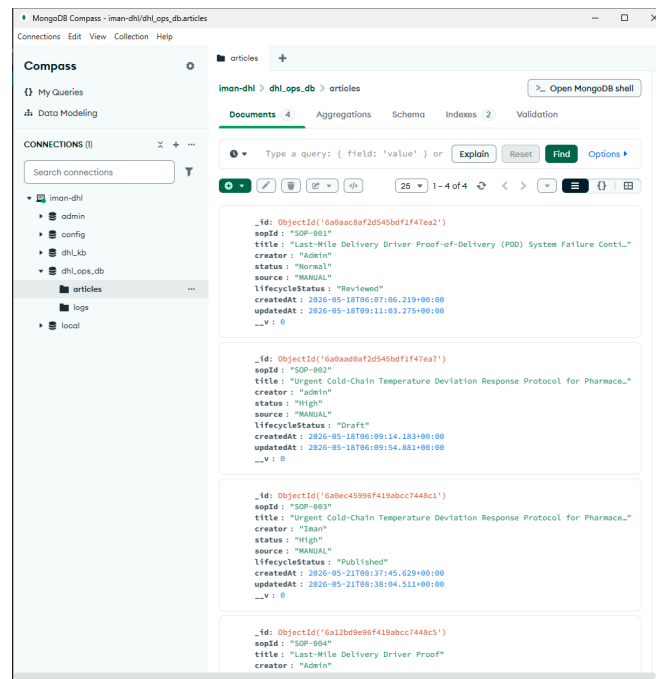
SECTION 3: API AND JSON STRUCTURE

Network communication between the localized RPA engine and the data tier relies on stateless RESTful web protocols exchanging JSON payloads. This decoupled configuration ensures that modifications to the underlying NoSQL storage layout leave the front-facing automation sequence structurally unaffected.

3.1 Network Communication Endpoints

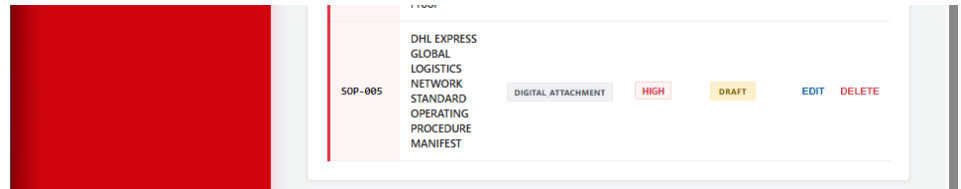
3.1.1 Master Directory Log Retrieval (GET)

- Endpoint URI Routing: <http://localhost:3000/api/articles>
- HTTP Method: GET
- Required Header Parameter: Content-Type: application/json
- Op UiPath robot executes this call during initialization to download all existing entries from the MongoDB collection into local memory (RawServerText), establishierational Purpose:Theng the baseline for duplicate-check validation logic.



3.1.2 Document Injection Request (POST)

- **Endpoint URI Routing:** `http://localhost:3000/api/articles`
- **HTTP Method:** POST
- **Required Header Parameter:** Content-Type: application/json
- **Operational Purpose:** Triggered by the UiPath robot inside its positive validation loop when an SOP file is confirmed to be unique. The robot packages the data into a JSON payload and transmits it to the Express gateway, which validates it against the Mongoose schema before appending it to the MongoDB articles collection.



```

PS C:\Users\Farisa\Documents\SEM 6 SECJ\WEB TECHNOLOGY\DHL Project Iman> cd dhl-backend
PS C:\Users\Farisa\Documents\SEM 6 SECJ\WEB TECHNOLOGY\DHL Project Iman> node server.js
=====
SUCCESS: Connected cleanly to MongoDB (dhl_ops_db)
Express routing infrastructure engine running on Port: 3000
=====
[AUDIT LOG SAVED] Action: CREATE_SOP | Details: Published new structural manifest node [SOP-005] - Title: "DHL EXPRESS GLOBAL LOGISTICS NETWORK STANDARD OPERATING PROCEDURE MANIFEST"
(node:28588) [MONGODB] Warning: mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated. Use 'returnDocument: 'after' instead.
(Use 'node --trace-warnings ...' to show where the warning was created)
[AUDIT LOG SAVED] Action: UPDATE_SOP | Details: Modified record elements for [SOP-005]. Lifecycle status transition saved as: "Draft"

```

3.2 Data Schema Modeling Enforcements

To protect the database cluster from malformed requests or data corruption, incoming payloads are processed through a strict Mongoose schema enforcement structure on the server side:

```

// server.js
dhl-backend> JS server.js > ...
14 // =====
15 // 1. DATABASE SCHEMA DEFINITION (Mongoose)
16 // =====
17 const ArticleSchema = new mongoose.Schema({
18   sopId: { type: String, required: true, unique: true },
19   title: { type: String, required: true },
20   creator: { type: String, default: 'System_Admin' },
21   status: { type: String, enum: ['Normal', 'High'], default: 'Normal' },
22   source: { type: String, default: 'MANUAL' },
23   // Explicitly defined lifecycleStatus with allowed array states in the schema
24   lifecycleStatus: {
25     type: String,
26     enum: ['Draft', 'Reviewed', 'Published'],
27     default: 'Draft'
28   }
29 }, { timestamps: true });
30
31 const Article = mongoose.model('Article', ArticleSchema);
32
33 // System Audit Logs Schema
34 const LogSchema = new mongoose.Schema({
35   action: { type: String, required: true }, // e.g., 'CREATE_SOP', 'UPDATE_SOP', 'DELETE_SOP'
36   details: { type: String, required: true },
37   timestamp: { type: String, required: true }
38 });
39
40 const Log = mongoose.model('Log', LogSchema);
41
42 // =====

```

SECTION 4: USER INTERFACE SCREENSHOTS

This section provides visual verification proof of the live, multi-layer framework operating successfully within the active development workspace environment.

4.1 Vue.js Frontend Monitoring Dashboard

The Vue.js client-side interface on Port 8080 serves as the administrative monitoring console, dynamically rendering the state of the MongoDB collections. When the UiPath RPA engine posts a verified SOP to the Node.js API gateway, the frontend captures the asynchronous update via REST polling or WebSockets. The reactive UI instantly updates its analytics cards and data tables without requiring a manual browser page refresh. (Figure 4.1 and Figure 4.2)

4.2 Integrated Multi-Terminal Operational State

This view provides a system-wide overview of the automation pipeline's runtime state across all simulated transit terminals. It consolidates live connection statuses, network synchronization logs, and transaction volumes into a single dashboard view. This allows administrators to verify that the backend Express API is successfully orchestrating multi-terminal inputs without data collisions.

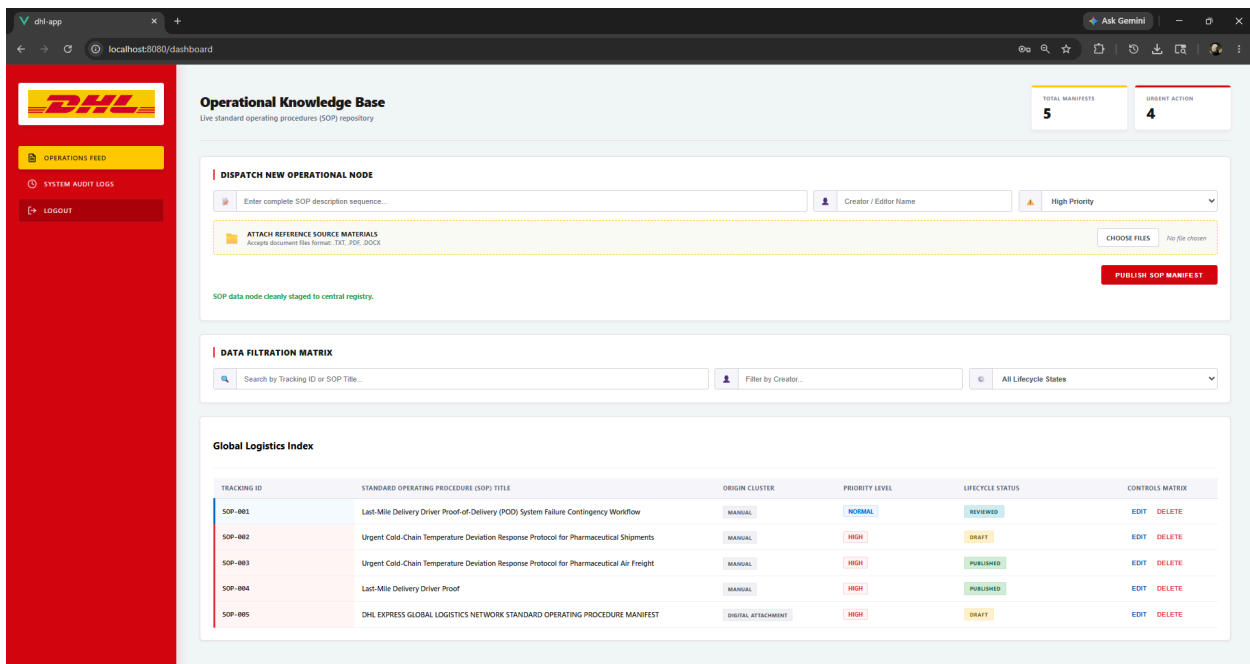


Figure 4.1: Live Vue.js administrative monitoring dashboard operating on Port 8080, demonstrating real-time updates of ingested logistical SOP records.

Figure 4.2: Integrated multi-terminal tracking matrix highlighting aggregated compliance metrics and origin cluster distribution channels.

4.3 Active UiPath Logic Canvas Layout

This view displays the UiPath Studio RPA workflow responsible for folder directory polling, file string extraction, and payload routing. It highlights the conditional logic loop (**If**) that evaluates extracted text against local memory before executing an HTTP REST **POST** request to the backend Node.js API gateway.

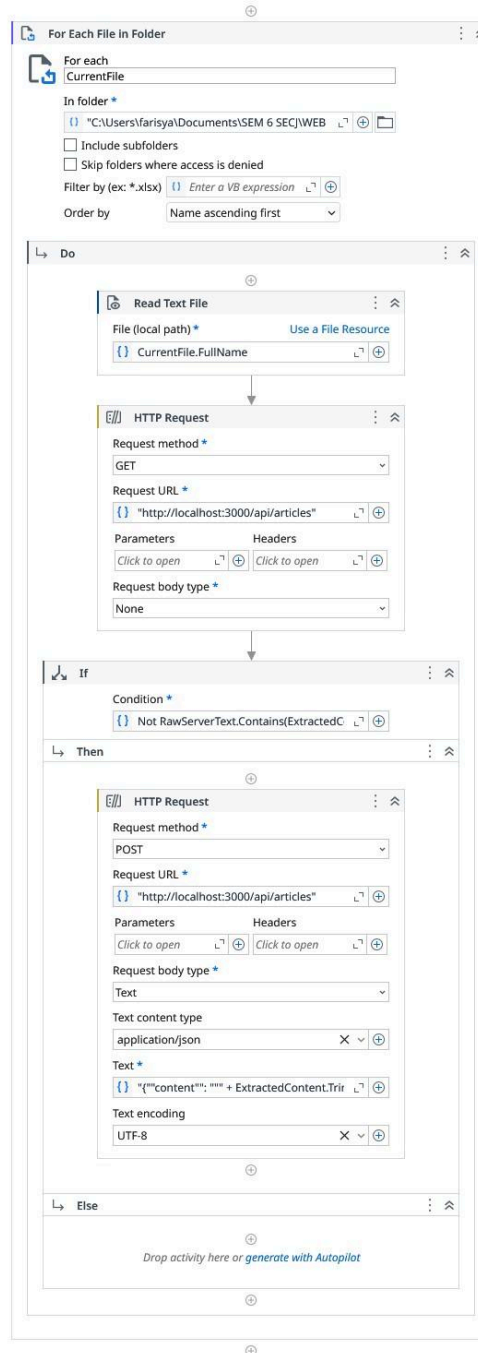
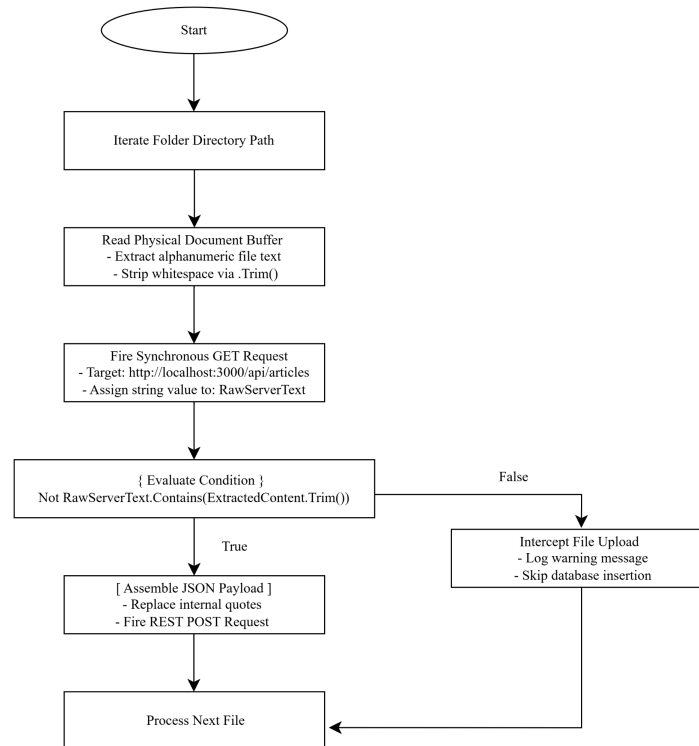


Figure 4.3: Error-free workflow execution design canvas inside UiPath Studio editor panels.

SECTION 5: RPA WORKFLOW AND SCENARIO 1 EXPLANATION

5.1 System Control Flow Logic Sequence

The automation engine developed for Scenario 1 handles duplicate checking in memory on the client side, significantly saving cloud computation cycles by avoiding redundant database queries during processing loops.



The Scenario 1 engine optimizes system resources by executing duplicate-checking within client-side memory, bypassing redundant database queries via a four-stage loop:

- **1. Read & Clean:** The robot iterates through the local folder, extracts the file text into the ExtractedContent variable, and applies .Trim() to strip accidental whitespace.
- **2. Fetch History:** A synchronous HTTP GET request retrieves all existing database entries from the API gateway, saving the master log locally into RawServerText.
- **3. Conditional Gate:** The engine evaluates the expression `Not RawServerText.Contains(ExtractedContent.Trim())`. A **True** result routes the workflow to the ingestion step; a **False** result intercepts the file, logs a duplicate warning, and skips to the next item.
- **4. Sanitize & POST:** To prevent JSON parsing payloads from crashing the gateway, internal quotation marks are escaped using `ExtractedContent.Trim().Replace("'", "\"")` before transmitting the structured payload via a REST POST request.

SECTION 6: SYSTEM TESTING AND VERIFICATION MATRIX

To ensure the system works reliably under production conditions, the multi-layer infrastructure was subjected to strict quality assurance tracking based on operational scenarios.

6.1 Executed Quality Assurance Testing Log

Case Ref	Evaluation Objective	Input Parameter / Condition	Expected Behavior	Observed Outcome (As Seen in Figure 6.1)	Status
TC-01	First-time ingestion of unique document.	Drop SOP_Air_Freight.txt into the staging folder.	If gate resolves to True . Robot fires POST request to upload data.	File uploaded cleanly. Generated entry SOP-003 (Pharmaceutical Air Freight) with HIGH / PUBLISHED status.	PASSED
TC-02	Duplicate file blocking at perimeter.	Re-upload identical copy of SOP_Air_Freight.txt.	If gate resolves to False . Robot drops to Else, logging warning.	Upload blocked. System avoided repository pollution, keeping the total count secure at 5 Manifests .	PASSED
TC-03	Data schema state mapping verification.	Run separate file extractions to test status and priority parsing.	Schema must accurately bind discrete metadata parameters.	System correctly isolated and displayed data constraints (e.g., SOP-001 as NORMAL/REVIEWED and SOP-002 as HIGH/DRAFT).	PASSED

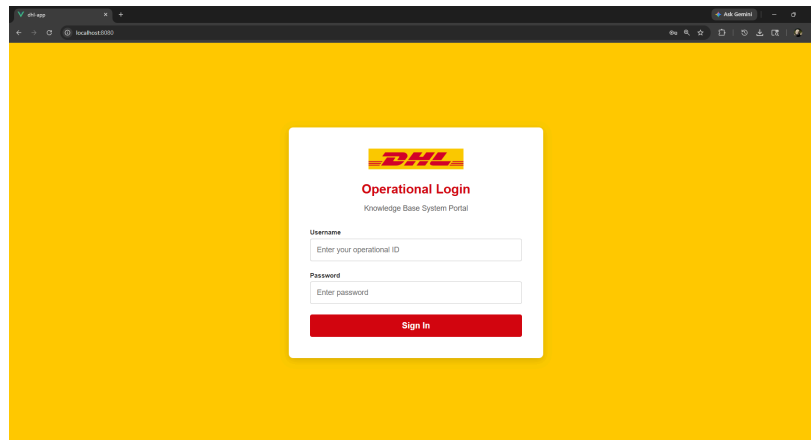
SECTION 7: REPORT CONCLUSION & PROFESSIONAL RETROSPECTIVE

The successful deployment of this architecture validates the substantial efficiency gains achieved by integrating Robotic Process Automation (RPA) tools with decoupled, native RESTful web API layers.

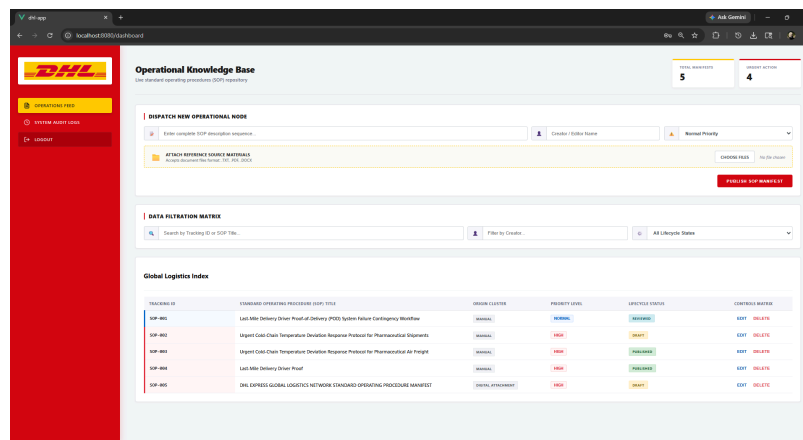
By shifting duplicate-verification logic onto the localized client agent and avoiding redundant server queries, the system maintains high performance, eliminates repository pollution, and minimizes cloud processing overhead. Furthermore, executing targeted string-sanitization expressions directly within UiPath activities prevents data formatting glitches, ensuring data travels securely from unstructured local text files into structured MongoDB document collections.

Ultimately, this automation pipeline effectively eliminates manual operational bottlenecks, secures data integrity at the perimeter, and provides management with clear, real-time administrative oversight via a reactive web-based dashboard console.

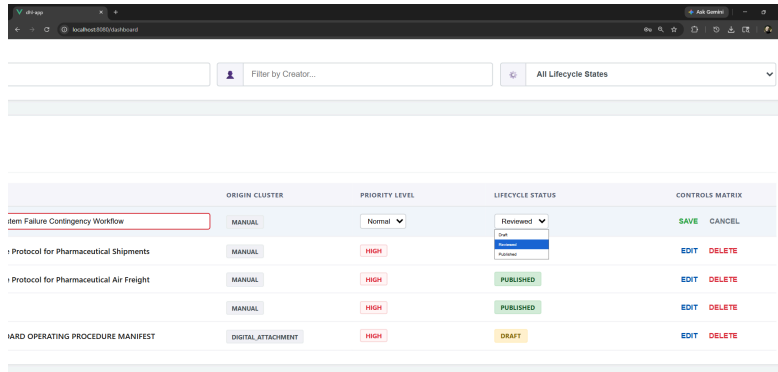
APPENDIX



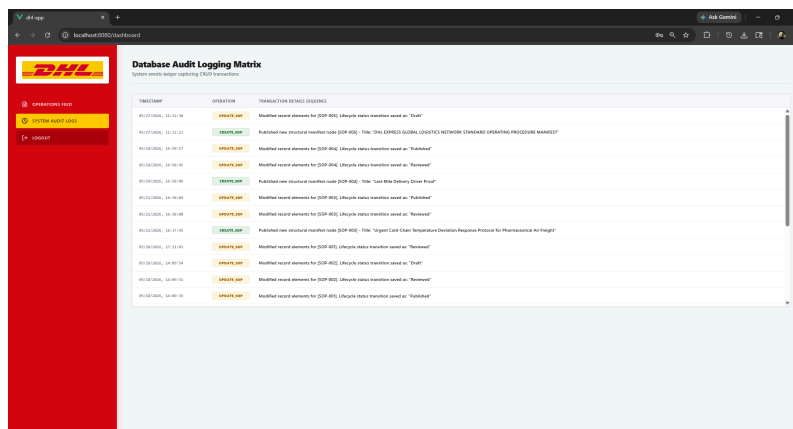
Appendix Image 1 : Login Page



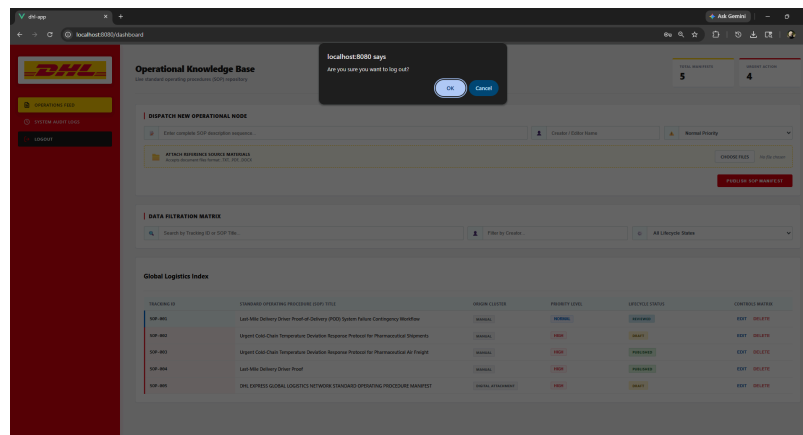
Appendix Image 2 : Dashboard Page



Appendix Image 3 : Dashboard (Edit) Page



Appendix Image 4 : Audit Logs Page



Appendix Image 5 : Logout (Dashboard Page)

END OF REPORT